# CS260 | Data Structures
# Written Assignment 1

**replace this text with your name and student id**

## 1 Questions

Question 1 : 35 points

For each of the following code snippets:

- Give a tight bound on the runtime of the snippet, and
- briefly explain your bound.

If the runtime is asymptotically different in different cases, give a separate bound for each case and briefly explain what the cases are, e.g. "insertion sort runs in $\Theta(n)$ time in the best case, when the list is already sorted, because...".

(a) [7 points] `foo`

```
/* L is array of n integers */
function foo(L, n)
{
  a = 0
  while (a < n) {
    b = a
    x = 0
    while (b < n) {
      x = x + L[b]
      b++
    }
    L[a] = x
    a++
  }
  return L
}
```

**Solution:** Your answer should replace this text

(b) [7 points] `baz`

```
/* L is a list of n integers */
function baz(L, n) {
  for (i = 0; i < n; i++) {
```

```
    if (L[i] == 7) {
      return true
    }
  }
  return false
}
```

**Solution:** Your answer should replace this text

(c) [7 points] `bar`

```
/* a, b, c, d are all integers */
function bar(a, b, c, d) {
  if (a < b) {
    if (c < d) {
      if (a < c) {
        return a
      } else {
        return c
      }
    } else {
      if (a < d) {
        return a
      } else {
        return d
      }
    }
  } else {
    if (c < d) {
      if (b < c) {
        return b
      } else {
        return c
      }
    } else {
      if (b < d) {
        return b
      } else {
        return d
      }
    }
  }
}
```

**Solution:** Your answer should replace this text

(d) [7 points] `qux`

```
/* n is a positive integer */
```

```
function qux(n) {
  for (i = 0; i < n; i++) {
    for (j = 0; j < n; j++) {
      for (k = 0; k < n; k++) {
        for (l = 0; l < 8; l++) {
          print(i * (j + k) - l)
        }
      }
    }
  }
}
```

**Solution:** Your answer should replace this text

(e) [7 points] `xyzzy`

```
/* n is a positive integer */
function xyzzy(n) {
  x = 2
  for (i = 0; i < n; i++) {
    x = x * 7
  }
  for (j = 0; j < 2*n; j++) {
    x = x / 3
  }
  return x
}
```

**Solution:** Your answer should replace this text

Question 2 : 20 points

Order the following functions from least to greatest growth rate, such that for any two functions $f$ and $g$, if $f$ comes before $g$, then $f(n) = O(g(n))$:

- $f_1(n) = n^n$
- $f_2(n) = n^{0.00001}$
- $f_3(n) = n^2$
- $f_4(n) = \lg(n)$
- $f_5(n) = 2^n$

**You must prove each ranking**, but you may assume the transitive property is true, which says that if $f(n) = O(g(n))$ and $g(n) = O(h(n))$, then $f(n) = O(h(n))$.

This means that if the functions were $f, g, h$ and you gave the order $f, g, h$, then you'd only need to prove that $f(n) = O(g(n))$ and $g(n) = O(h(n))$, as you may assume that this implies $f(n) = O(h(n))$.

**You must fully simplify your limit ratios to receive credit.** If you haven't simplified your limit ratio to 0, $\infty$, or a positive constant, you're not finished simplifying.

**Solution:** Your answer should replace this text

Question 3 : 15 points

Consider the following function `compute`:

```
1  /* n is a positive integer */
2  function compute(n)
3  {
4    if (n == 0)
5    {
6      return 0
7    }
8    x = 0
9    for (i = 1; i <= n; i++)
10   {
11     x = x + i
12   }
13   return compute(n - 1) + x
14 }
```

(a) [5 points] Give a recurrence that represents the runtime of `compute`.

**Solution:** Your answer should replace this text

(b) [10 points] Solve your recurrence using the substitution method, then give a tight bound on the runtime of `compute`.

**Solution:** Your answer should replace this text

Question 4 : 30 points

Professor Holmes has come up with a new sorting algorithm. He calls it **Trimerge Sort** and "claims" that it is asymptotically faster than **Merge Sort**, despite the fact both the algorithms operate using similar logic. But, unlike Merge Sort, Trimerge Sort splits the input array into *(roughly)* 3 equal parts at each step of the recursion as long as the array is splittable *(i.e., has at least 3 elements)*. Trimerge Sort's merge subroutine, similar in principle to the one used by Merge Sort, takes 3 sorted subarrays and merges them to produce a single sorted array. Given all of this, answer the following questions.

(a) [5 points] In Merge Sort, the **merge subroutine** makes $n$ comparisons to merge 2 arrays of size $n/2$, which takes $\Theta(n)$ time. How many comparisons (in exact numbers) will the merge subroutine of Trimerge Sort make to merge 3 arrays of size $n/3$? What would be the $\Theta(\cdots)$ bound on the running time for merge subroutine?

**Solution:** Your answer should replace this text

(b) [5 points] What is the recurrence relation for the Trimerge Sort algorithm? Also explain it in plain English what each term in the recurrence relation stands for.

**Solution:** Your answer should replace this text

(c) [8 points] Prove the $\Theta(\cdots)$ bound on the running time of the Trimerge Sort algorithm using the **Recursion Tree Method** covered in class.

**Solution:** Your answer should replace this text

(d) [8 points] Prove the $\Theta(\cdots)$ bound on the running time of the Trimerge Sort algorithm using **Equation Expansion** method over the recurrence relation you came up with in part (b).

**Solution:** Your answer should replace this text

(e) [4 points] Is Trimerge Sort asymptotically faster than Merge Sort? Why/why not? Reason through your answer using **a mathematical proo**f.

*(**Hint:** Use the limit method demonstrated in class as well as the recitation on Asymptotic Analysis & Recurrence Relations)*

**Solution:** Your answer should replace this text